HƯỚNG DẪN SINH VIÊN THỰC HIỆN CT – PBL – AI

I. Mục tiêu chung

Trang bị cho sinh viên năng lực tư duy giải quyết vấn đề, tư duy logic và khả năng giao tiếp kỹ thuật theo hướng **nâng cao tư duy**. Đây là khung phương pháp tổng hợp xây dựng trên nền tảng:

- Computational Thinking (CT): 4 thành phần nền tảng rèn luyện tư duy logic.

- AI Reflection: Năng lực làm chủ AI, phát hiện ảo giác (hallucination) và đảm bảo "Human Delta" — giá trị sáng tạo của con người mà AI không thể thay thế.

II. Định hướng phương pháp CT + PBL

CT + PBL bao gồm 3 lớp tích hợp chặt chẽ:

2.1. Computational Thinking (CT) - Nền tảng

- Decomposition: Phân rã bài toán lớn thành các phần nhỏ.

- Pattern Recognition: Nhận diện mô hình tương đồng để tái sử dụng giải pháp.

- Abstraction: Khái quát hóa vấn đề, lược bỏ chi tiết thừa để tạo giải pháp tổng quát.

- Algorithms: Xây dựng quy trình thực hiện tuần tự.

2.2. Project-Based Learning (PBL)

Project-Based Learning - Học tập qua dự án (PBL) là một mô hình lấy người học làm trung tâm. Trong đó, sinh viên lĩnh hội kiến thức và kỹ năng thông qua việc dành một khoảng thời gian dài để nghiên cứu, phản hồi và giải quyết một câu hỏi, một vấn đề hoặc một thử thách mang tính thực tiễn cao.

Khác với phương pháp truyền thống, dự án trong PBL không phải là hoạt động bổ trợ sau bài học mà chính là phương thức truyền tải nội dung bài học.

Các thành phần cốt lõi của mô hình PBL:

- Vấn đề/Câu hỏi dẫn dắt (Problem/Driving Question): Phải đảm bảo tính thách thức, thực tế và phù hợp với chuẩn đầu ra của học phần.

- **Sự chủ động của người học (Student Voice & Choice)**: Sinh viên được quyền tham gia quyết định về phương pháp tiếp cận và hình thức thể hiện sản phẩm.

- **Kỹ năng**: Tập trung phát triển năng lực tư duy phản biện, làm việc nhóm, giao tiếp và quản lý dự án.

- **Quá trình cải thiện (Review & Revision)**: Thiết lập các cột mốc (milestones) để giảng viên phản hồi và sinh viên tự hiệu chỉnh trước khi có kết quả cuối cùng.

- **Sản phẩm công khai (Public Product)**: Kết quả dự án phải được trình bày trước hội đồng hoặc ứng dụng vào thực tế, tạo động lực về trách nhiệm và chất lượng công việc.

Trong các môn kỹ thuật lập trình – **Programming Fundamentals** với C/C++ hay Python (PRF19x, PFP191 và PCP291), sinh viên sẽ học thông qua việc xây dựng một chương trình quản lý, đây là một dự án nhỏ, giúp các bạn áp dụng kiến thức từ cơ bản đến nâng cao: kiểu dữ liệu, biến, hàm, thư viện, File, quản lý lỗi vào thực tế.

2.3. AI Reflection - Tư duy phản biện AI

Sinh viên được khuyến khích dùng AI và sử dụng **AI Audit Log** (Nhật ký sử dụng AI) để:

- Giải thích lý do tại sao chọn hoặc điều chỉnh gợi ý từ AI.

- Phát hiện các lỗi sai logic hoặc thông tin giả (fabrication) từ AI.

- Minh chứng sự tham gia của bản thân (Human Delta) trong quá trình thực hiện.

2.4. Sản phẩm cuối cùng

- Một chương trình phần mềm – dự án hoàn chỉnh.

- Báo cáo dự án theo mẫu chuẩn.

- Slide thuyết trình nhóm.

- Các chứng chỉ/ chứng nhận theo chỉ định đã được Nhà trường phê duyệt

III. Tổ chức nhóm và trách nhiệm triển khai

3.1. Tổ chức nhóm

- Mỗi nhóm tối đa **04 sinh viên**.

- Nhóm trưởng điều phối công việc và liên lạc với giảng viên.

- Tất cả thành viên phải tham gia đầy đủ các giai đoạn và trình bày ở tuần cuối.

Gợi ý phân công

Tuần	Hoạt động	Phân công
Tuần 1	Kickoff	Chọn nhóm, chọn đề tài chủ đề
Tuần 2	Phân tích yêu cầu	Cả nhóm thảo luận, nhóm trưởng tổng hợp

Tuần	Hoạt động	Phân công
Tuần 3 - 4	Thiết kế thuật toán & Flowchart	Mỗi thành viên phụ trách 1 chức năng
Tuần 5 - 6	Cài đặt hàm cơ bản	Chia đều các chức năng (Add, Update, Delete…)
Tuần 7	Thiết kế OOP & STL	2 người viết class, 2 người các chức năng khác liên quan
Tuần 8	File I/O & Testing	1 người viết đọc/ghi file, 1 người viết test cases, 1 người test, 1 người hoàn chỉnh báo cáo
Tuần 9 - 10	Báo cáo & Thuyết trình	Phân công viết file thuyết trình, tất cả cùng trình bày

3.2. Lộ trình thực hiện theo tuần

Giai đoạn	Tuần	CT Component	Hoạt động chính	Sản phẩm & AI Reflection
Kickoff	1	Kickoff	Chọn nhóm, đề tài, phân công vai trò	Danh sách thành viên, đề tài
Khởi động	2	Decomposition + Thinking	Phân tích yêu cầu, decomposition thành module	- Bảng phân rã chức năng (Requirement Analysis) - Sơ đồ mô tả bài toán. Bảng so sánh "Phác thảo ban đầu" vs "Gợi ý của AI".
Biểu diễn Insight	Tuần 3 - 4	Abstraction + Algorithms + Research	Viết Pseudocode, Vẽ sơ đồ thuật toán cho Add, Search, Save/Load…	- Bộ flowchart hoàn chỉnh, pseudo code - Giải thuật, tài liệu tham khảo. - Reflection: Tại sao chọn/hiệu chỉnh gợi ý AI

Giai đoạn	Tuần	CT Component	Hoạt động chính	Sản phẩm & AI Reflection
	Tuần 5 - 6	Decomposition + Abstraction	Cài đặt hàm cơ bản (add, update, delete…)	Code chạy được các chức năng cơ bản
Mô hình hóa	Tuần 7	Decomposition + Abstraction	Thiết kế các struct/class liên quan, các prototype, hàm dùng chung	- UML + code các module - AI Audit Log (liệt kê prompt quan trọng và phân tích lỗi AI) – Tùy chọn.
Project-based	Tuần 8	Tổng hợp (CT + AI + PBL)	Thêm các hàm save/load, exception handling, viết test cases	- File I/O + báo cáo test - Bản thảo Report
Trình bày	Tuần 9 - 10	Report & Presentation	Hoàn chỉnh báo cáo, chuẩn bị slide, thuyết trình	AI Audit Log hoàn chỉnh (opt) + Slide + Mini Report + Source code.

IV. Hướng dẫn thực hiện các hoạt động

4.1. Requirement Analysis & Decomposition – Phân tích yêu cầu và phân rã

4.1.1. Phân tích dữ liệu, kiểu dữ liệu

- Trả lời câu hỏi để xác định dữ liệu, kiểu dữ liệu, cấu trúc dữ liệu, ràng buộc.

Bước 1: Xác định thực thể và thuộc tính (Input Discovery)

Mục tiêu: Giúp sinh viên xác định những thông tin cần thiết để giải quyết bài toán.

1. Để giải quyết vấn đề này, nếu bạn làm thủ công trên giấy, bạn cần những mẩu tin hay con số cụ thể nào từ người dùng?

2. Chương trình cần nhận bao nhiêu thông số để có thể bắt đầu tính toán/xử lý? Hãy liệt kê tên của từng thông số đó.

3. Trong các dữ liệu đã liệt kê, thông tin nào sẽ không bao giờ thay đổi (hằng số) và thông tin nào sẽ thay đổi trong quá trình thực thi (biến số)?

Bước 2: Xác định kiểu dữ liệu (Data Typing)

Mục tiêu: Giúp sinh viên chọn kiểu dữ liệu tối ưu cho bộ nhớ và logic.

1. Dữ liệu này là số (đếm được, đo lường được), là chữ (tên gọi, mô tả) hay là một trạng thái Đúng/Sai (Boolean)?

2. Nếu là dữ liệu số, nó có bao giờ xuất hiện phần thập phân không? Phạm vi giá trị lớn nhất và nhỏ nhất mà nó có thể đạt tới là bao nhiêu? (Câu hỏi này giúp chọn giữa kiểu số int, long hay kiểu thực float hoặc double).

3. Thông tin này chỉ là một ký tự duy nhất (như mã loại, phái) hay là một chuỗi ký tự dài (như họ tên, địa chỉ)?

Bước 3: Xác định cấu trúc dữ liệu (Data Structure)

Mục tiêu: Giúp sinh viên nhận diện khi nào cần dùng biến đơn hoặc tập hợp dữ liệu.

1. Bạn cần lưu trữ một giá trị duy nhất hay một danh sách nhiều giá trị cùng loại (ví dụ: một điểm số hay danh sách điểm của cả lớp)?

2. Các dữ liệu này có liên quan chặt chẽ với nhau để tạo thành một đối tượng cụ thể không (ví dụ: Mã SV, Tên SV, Ngày sinh)? Nếu có, liệu có nên nhóm chúng lại vào một cấu trúc (record/struct/class) hay không?

Bước 4: Ràng buộc và điều kiện dữ liệu (Data Constraints)

Mục tiêu: Giúp sinh viên chuẩn bị cho việc kiểm soát lỗi (Validation).

1. Dữ liệu đầu vào có những giới hạn thực tế nào không? (Ví dụ: Tuổi không được âm, điểm số phải từ 0 đến 10).

2. Chuyện gì xảy ra nếu người dùng không nhập gì cả? Dữ liệu này có bắt buộc phải có giá trị ngay từ đầu hay có thể để trống?

4.1.2. Phân tích chia nhỏ các chức năng

Trả lời các câu hỏi mẫu để xác định chức năng liên quan.

Bước 1: Nhóm câu hỏi xác định mục tiêu cốt lõi (Core Functions)

Mục tiêu: Tìm ra những chức năng "sống còn", nếu không có nó thì dự án không còn ý nghĩa.

1. Chức năng quan trọng nhất mà người dùng phải thực hiện được trên hệ thống này là gì?

2. Người dùng tìm đến phần mềm/ứng dụng này để giải quyết vấn đề cụ thể nào?

3. Sau khi sử dụng hệ thống, người dùng sẽ nhận được kết quả hoặc sản phẩm cuối cùng là gì (ví dụ: một báo cáo, một đơn hàng đã thanh toán, một tệp tin đã chuyển đổi, …)?

Bước 2: Nhóm câu hỏi xác định vai trò người dùng (User Roles)

Mục tiêu: Phân quyền và xác định các luồng chức năng khác nhau.

1. Người dùng nào sẽ tương tác với hệ thống?

2. Chức năng nào người dùng thường được sử dụng?

Bước 3: Nhóm câu hỏi về quy trình nghiệp vụ (Business Logic)

Mục tiêu: Xác định các bước xử lý dữ liệu bên trong.

1. Để hoàn thành mục tiêu [X], người dùng cần đi qua những bước cụ thể nào? Thứ tự các bước đó ra sao?

2. Nếu người dùng nhập sai thông tin hoặc hệ thống gặp lỗi ở bước [Y], chương trình cần phải làm gì (ví dụ: thông báo lỗi, cho phép nhập lại, hay tự động khôi phục dữ liệu)?

3. Có những quy tắc nghiệp vụ nào cần áp dụng không? (Ví dụ: Chỉ được rút tiền nếu số dư > 50.000 VNĐ; Chỉ được đăng ký môn học nếu đã học môn tiên quyết).

Bước 4: Nhóm câu hỏi về quản lý và báo cáo (Management & Reporting)

Mục tiêu: Xác định các chức năng hỗ trợ và giám sát.

1. Người dùng có cần các chức năng Thêm, Xóa, Sửa, Tìm kiếm (CRUD) cho các thực thể dữ liệu không?

2. Hệ thống có cần xem các báo cáo tổng hợp hay biểu đồ thống kê về hoạt động của dự án không? Nếu có, đó là những chỉ số nào?

Bước 5: Nhóm câu hỏi về phi chức năng (Non-functional Requirements)

Mục tiêu: Xác định các yếu tố chất lượng ảnh hưởng đến chức năng.

1. Dự án này cần chạy trên nền tảng nào?

2. Giao diện cần Menu điều hướng thế nào?

4.2. Algorithm Design & Flowcharts - Thiết kế thuật toán & Flowchart

- Xác định điểm bắt đầu, điểm kết thúc, dữ liệu đầu vào, dữ liệu sau khi xử lý, dữ liệu cần trả về và từng bước xử lý và các điều kiện liên quan (rẽ nhánh, điều kiện dừng vòng lặp) trong từng function, hàm.

- Xây dựng Pseudo code như Guideline.

- Minh họa với Flowchart

4.3. Implementation of Basic Functions – Viết các hàm cơ bản.

- Dựa vào các thư viện, cấu trúc dữ liệu, code mẫu để xây dựng các hàm cơ bản của dữ án.

- Viết các hàm cơ bản dựa trên decomposition.

- Liên hệ các kiểu dữ liệu trừu tượng Abstract Data-type (ADT): Record, Array, Linker-List, để có giải pháp code phù hợp

4.4. Struct & Module Design/Object-Oriented Design

- Dựa vào ngôn ngữ lập trình tương ứng, tổ chức dự án thành các module và các hàm riêng biệt phù hợp.

- Với ngôn ngữ lập trình hỗ trợ OOP như C++, Python, vận dụng các tính chất của OOP xây dựng các class phù hợp.

- Tách thành các file riêng biệt, mỗi file tổ chức cho một module hoặc class.

4.5. File I/O & Testing

- **File I/O**: Đề xuất giải pháp lưu dữ liệu vào file, đọc lại khi khởi động.

- **Exception Handling**: Thông báo lỗi khi file không mở được.

- **Test Cases**: Dự thảo các tình huống lỗi và thực hiện kiểm thử các chức năng.

4.6. Báo cáo

Cấu trúc báo cáo theo mẫu

- **Introduction**: Mô tả dự án, nhóm.

- **Requirement Analysis & Decomposition**: Bảng phân rã chức năng.

- **Algorithm Design & Flowcharts**: Sơ đồ thuật toán.

- **Implementation**: Code cơ bản.

- **Struct / OOP Design**: UML + class.

- **File I/O & Testing**: Test cases + kết quả.

- **Experimental Results**: Kết quả chụp màn hình chạy các chức năng

- **Appendix**: File dữ liệu và **AI Audit Log**

V. Rubric chấm điểm theo công bố của Syllabus